

Module 7: Cursors in PL/SQL

Duration

8–10 Hours (Theory + Laboratory Practice)

Learning Outcomes

After completing this module, students will be able to:

- Understand the concept of cursors in PL/SQL.
 - Explain why cursors are required.
 - Differentiate between implicit and explicit cursors.
 - Use cursor attributes.
 - Develop programs using explicit cursors.
 - Implement Cursor FOR Loops.
 - Create parameterized cursors.
 - Process multiple records efficiently.
 - Develop real-world database applications using cursors.
-

7.1 Introduction to Cursors

What is a Cursor?

A **Cursor** is a memory area created by Oracle to store the result set of an SQL query.

It acts as a **pointer** that allows PL/SQL to retrieve and process one row at a time from the query result.

Definition

A cursor is a mechanism used by PL/SQL to retrieve, manage, and process rows returned by an SQL statement one at a time.

Why Do We Need Cursors?

Suppose the **EMPLOYEE** table contains the following records.

EMP_ID EMP_NAME SALARY

101	Rahul	45000
102	Amit	50000
103	Priya	60000
104	Neha	55000

Suppose we want to display every employee's name and salary.

Without a cursor, PL/SQL cannot process multiple rows individually.

A cursor allows us to:

- Retrieve one row.
 - Process it.
 - Move to the next row.
 - Continue until all rows are processed.
-

Real-Life Analogy

Imagine a teacher checking examination papers.

Instead of checking all papers simultaneously, the teacher:

Pick one answer sheet

↓

Evaluate it

↓

Record marks

↓

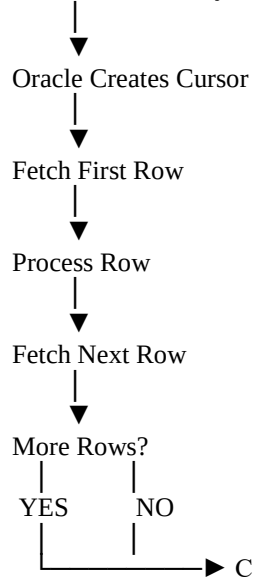
Pick the next answer sheet

↓

Repeat until all papers are evaluated
A cursor behaves in exactly the same way.

How Cursor Works

Execute SQL Query



Types of Cursors

Oracle supports two types of cursors.

Cursor Type	Description
-------------	-------------

Implicit Cursor	Automatically created by Oracle
-----------------	---------------------------------

Explicit Cursor	Created by the programmer
-----------------	---------------------------

7.2 Implicit Cursor

What is an Implicit Cursor?

Whenever a PL/SQL program executes:

- INSERT
- UPDATE
- DELETE
- SELECT INTO

Oracle automatically creates a cursor.

The programmer does not declare or manage it.

Example

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    total NUMBER;
```

```
BEGIN
```

```
    SELECT COUNT(*)
```

```

    INTO total

    FROM employee;

    DBMS_OUTPUT.PUT_LINE('Total Employees = '||total);

END;
/

```

Oracle automatically:

- Opens the cursor.
- Executes the query.
- Fetches the record.
- Closes the cursor.

Implicit Cursor Attributes

Oracle provides four attributes.

Attribute	Purpose
SQL%FOUND	TRUE if rows affected
SQL%NOTFOUND	TRUE if no rows affected
SQL%ROWCOUNT	Number of affected rows
SQL%ISOPEN	Always FALSE (implicit cursor closes automatically)

Example

```

DECLARE

rows_updated NUMBER;

BEGIN

UPDATE employee

SET salary=salary+2000

WHERE dept_id=10;

rows_updated:=SQL%ROWCOUNT;

DBMS_OUTPUT.PUT_LINE(rows_updated||' Employees Updated');

END;
/

```

Example

Suppose 6 employees belong to Department 10.
Output
6 Employees Updated

7.3 Explicit Cursor

Why Explicit Cursor?

Suppose the SQL query returns many rows.

Example

```
SELECT *
```

```
FROM employee;
```

A SELECT INTO statement cannot process multiple rows.
Therefore, we use Explicit Cursor.

What is Explicit Cursor?

An Explicit Cursor is declared and managed by the programmer.

The programmer controls:

- Opening the cursor
 - Fetching rows
 - Closing the cursor
-

Steps in Explicit Cursor

There are four steps.

Step 1

Declare Cursor

Step 2

Open Cursor

Step 3

Fetch Records

Step 4

Close Cursor

Cursor Life Cycle

DECLARE Cursor

|



OPEN Cursor

|



FETCH Row

|



Process Data

|



More Rows?

YES —————> FETCH Again

NO

|



CLOSE Cursor

Syntax

DECLARE

CURSOR cursor_name IS

SELECT statement;

BEGIN

OPEN cursor_name;

LOOP

FETCH cursor_name INTO variables;

EXIT WHEN cursor_name%NOTFOUND;

processing statements;

END LOOP;

CLOSE cursor_name;

END;

/

Example

Display Employee Names

SET SERVEROUTPUT ON;

DECLARE

CURSOR emp_cursor IS

SELECT emp_name

FROM employee;

v_name employee.emp_name%TYPE;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor INTO v_name;

EXIT WHEN emp_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_name);

END LOOP;

```
CLOSE emp_cursor;
```

```
END;
```

```
/
```

```
Output
```

```
Rahul
```

```
Amit
```

```
Priya
```

```
Neha
```

Example

Display Employee Name and Salary

```
DECLARE
```

```
CURSOR emp_cursor IS
```

```
SELECT emp_name,salary
```

```
FROM employee;
```

```
v_name employee.emp_name%TYPE;
```

```
v_salary employee.salary%TYPE;
```

```
BEGIN
```

```
OPEN emp_cursor;
```

```
LOOP
```

```
FETCH emp_cursor
```

```
INTO v_name,v_salary;
```

```
EXIT WHEN emp_cursor%NOTFOUND;
```

```
DBMS_OUTPUT.PUT_LINE(v_name||' '||v_salary);
```

```
END LOOP;
```

```
CLOSE emp_cursor;
```

```
END;
```

```
/
```

Cursor Attributes

Explicit cursors also provide useful attributes.

Attribute	Description
-----------	-------------

%FOUND	Last fetch successful
--------	-----------------------

%NOTFOUND	No more rows
-----------	--------------

%ROWCOUNT	Number of fetched rows
-----------	------------------------

Attribute	Description
%ISOPEN	TRUE if cursor is open

Example

```
EXIT WHEN emp_cursor%NOTFOUND;
```

Example

```
DBMS_OUTPUT.PUT_LINE(emp_cursor%ROWCOUNT);
```

Displays the number of fetched records.

7.4 Cursor FOR LOOP

What is Cursor FOR LOOP?

Oracle provides a simplified way of processing cursors.

No need to:

- OPEN
- FETCH
- CLOSE

Oracle performs these operations automatically.

Syntax

```
FOR record IN cursor_name LOOP
```

```
statements;
```

```
END LOOP;
```

Example

```
DECLARE
```

```
CURSOR emp_cursor IS
```

```
SELECT emp_name,salary
```

```
FROM employee;
```

```
BEGIN
```

```
FOR emp_record IN emp_cursor LOOP
```

```
DBMS_OUTPUT.PUT_LINE(
```

```
emp_record.emp_name||
```

```
' Salary = '||
```

```
emp_record.salary);
```

```
END LOOP;
```

```
END;
```

```
/
```

Notice that:

No OPEN

No FETCH

No CLOSE

Oracle does everything.

Advantages

- Less code
 - Easy to understand
 - Fewer programming mistakes
 - Recommended for beginners
-

7.5 Parameterized Cursor

Why Parameterized Cursor?

Suppose we want employees of different departments.

Department 10

Department 20

Department 30

Instead of creating multiple cursors,
we pass department number as a parameter.

Syntax

CURSOR cursor_name(parameter datatype)

IS

SELECT ...

WHERE condition;

Example

DECLARE

CURSOR emp_cursor(dno NUMBER)

IS

SELECT emp_name

FROM employee

WHERE dept_id=dno;

v_name employee.emp_name%TYPE;

BEGIN

OPEN emp_cursor(20);

LOOP

FETCH emp_cursor INTO v_name;

EXIT WHEN emp_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_name);

END LOOP;

CLOSE emp_cursor;

END;
/

Practical Programs

Program 1

Display all employees.

Program 2

Display employee names with salary greater than ₹50,000.

Program 3

Calculate total salary of all employees using cursor.

DECLARE

CURSOR emp_cursor IS

SELECT salary

FROM employee;

v_salary NUMBER;

total NUMBER:=0;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor INTO v_salary;

EXIT WHEN emp_cursor%NOTFOUND;

total:=total+v_salary;

END LOOP;

CLOSE emp_cursor;

DBMS_OUTPUT.PUT_LINE(total);

END;
/

Program 4

Increase salary of every employee by 10%.

Program 5

Display department-wise employee list.

Implicit vs Explicit Cursor

Feature	Implicit Cursor	Explicit Cursor
Created By	Oracle	Programmer

Feature	Implicit Cursor	Explicit Cursor
Open Cursor	Automatic	Manual
Fetch Rows	Automatic	Manual
Close Cursor	Automatic	Manual
Suitable For	Single-row operations	Multi-row operations
Complexity	Simple	More flexible

Cursor FOR LOOP vs Explicit Cursor

Cursor FOR LOOP	Explicit Cursor
Automatic OPEN/FETCH/CLOSE	Manual management
Less coding	More coding
Beginner-friendly	Greater control
Less error-prone	More flexible

Real-World Applications

Cursors are extensively used in:

- Payroll Generation
 - Salary Processing
 - Student Result Processing
 - Bank Interest Calculations
 - Report Generation
 - Data Migration
 - Inventory Management
 - Monthly Billing
 - Insurance Claims
 - ERP Systems
-

Common Errors by Beginners

- Forgetting to CLOSE the cursor.
 - Omitting EXIT WHEN cursor%NOTFOUND, resulting in an infinite loop.
 - Fetching into an incorrect number of variables.
 - Attempting to fetch from a cursor that has not been opened.
 - Opening the same cursor multiple times without closing it.
-

Summary

In this module, we introduced **Cursors**, a fundamental feature of PL/SQL for processing query results. We explored the differences between implicit and explicit cursors, learned how to declare, open, fetch from, and close explicit cursors, used cursor attributes to monitor cursor state, simplified cursor handling with the Cursor FOR LOOP, and implemented parameterized cursors for reusable query logic. Cursors are indispensable for enterprise applications that need to process multiple rows of data efficiently.

Laboratory Exercises

1. Display all employee records using an explicit cursor.
2. Display employee names and salaries using a Cursor FOR LOOP.
3. Calculate the total salary of all employees.
4. Display employees belonging to a specified department using a parameterized cursor.
5. Update salaries and display the number of rows affected using SQL%ROWCOUNT.
6. Display the first five employee records using a cursor.
7. Count the number of employees in each department using cursors.
8. Generate a salary report for all employees.
9. Process student records to calculate grades using a cursor.
10. Create a report showing employees with salaries above the department average.

Viva/Interview Questions

1. What is a cursor in PL/SQL?
2. Why are cursors required?
3. Differentiate between implicit and explicit cursors.
4. Explain the life cycle of an explicit cursor.
5. What are the four cursor attributes?
6. What is the purpose of EXIT WHEN cursor%NOTFOUND?
7. What is a Cursor FOR LOOP, and how does it simplify cursor programming?
8. What is a parameterized cursor?
9. When should you use an explicit cursor instead of SELECT INTO?
10. Give three real-world applications where cursors are commonly used.